

plan

Web2 (JBPM) 和 Web3 (区块链) 异步事件驱动集成计划

项目概述

实现JBPM工作流与区块链的**异步事件驱动**集成。采用**混合方案**：保留现有CCProject合约用于资金管理，同时为每个项目铸造ERC-1155 dNFT作为数字凭证和收藏品。

核心架构变更

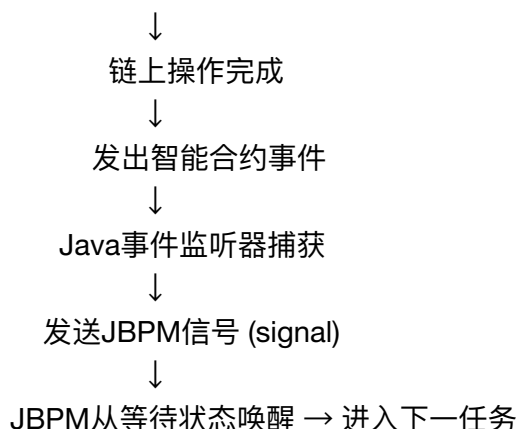
⚠ 关键架构差异

现有架构（同步）：

JBPM任务完成 → 立即执行区块链操作 → 返回结果 → 进入下一任务

新架构（异步事件驱动）：

JBPM任务完成 → 触发区块链操作 → JBPM进入等待状态



用户需求确认

- 架构模式: 异步事件驱动
- 工作流顺序: Value Estimation → **Bid Review** → **Funding** (需修改BPMN!)
- 通知机制: 区块链事件监听 + JBPM信号
- 等待机制: BPMN等待任务 (Wait Task)
- 投资转账: 后端代理转账 (Admin钱包代理执行)
- **NFT所有权**: Admin托管模式
- 投资者表示: NFT元数据记录

现有架构分析

现有JBPM工作流顺序（需修改）

当前BPMN文件顺序：

1. Value Estimation
2. **Funding** ← 当前在这里
3. **Bid Review** ← 当前在这里
4. Assign Restoration
5. Restoration Finished
6. Post Item
7. Auction
8. Auction Review

需要修改为：

1. Value Estimation
2. **Bid Review** ← 移到Funding之前
3. **Funding** ← 移到Bid Review之后
4. Assign Restoration
5. Restoration Finished
6. Post Item
7. Auction
8. Auction Review

现有区块链集成模式（同步）

```
// 当前模式：同步
processUtility.completeTask(taskId, user, params);
blockchainService.doSomething(); // 阻塞等待
projectListingRepository.save(listing);
```

新工作流程详细说明

完整的异步流程

1. Value Estimation → 铸造NFT

用户: 完成Value Estimation任务



后端: 完成JBPM任务 → 触发区块链操作（mintDNFT + launch）



JBPM: 进入"Waiting for NFT Minting"等待任务



智能合约: 铸造NFT → 发出DNFTCreated事件



事件监听器: 捕获事件 → 发送JBPM信号"nftMinted"



JBPM: 收到信号 → 进入Bid Review任务

2. Bid Review → 选择修复者

Admin: 完成Bid Review（选择修复者）



后端: 完成JBPM任务（无区块链操作）



JBPM: 直接进入Funding任务

3. Funding → 投资者转账

投资者: 提交投资请求（Web2界面）



后端: 代理执行链上转账（调用CCProject.acceptFunds）
+ 更新NFT投资者元数据



JBPM: 进入"Waiting for Investment Confirmation"



智能合约: 投资成功 → 发出InvestmentAccepted事件



事件监听器: 捕获事件 → 发送JBPM信号"investmentConfirmed"



JBPM: 收到信号 → 检查是否达到融资目标
如达到 → 进入Assign Restoration

4. Assign Restoration → 分配修复

Admin: 完成Assign Restoration任务



后端: 完成JBPM任务 → 触发链上assignRestoration + 更新NFT元数据



JBPM: 进入"Waiting for Restoration Assignment"



智能合约: 分配完成 → 发出RestorationAssigned事件



事件监听器: 发送信号 → JBPM进入Restoration Complete

5. Restoration Complete → 标记完成

修复者: 完成Restoration Complete任务



后端: 触发链上更新NFT元数据（修复完成状态）



JBPM: 等待确认 → 收到信号 → 进入Post Item

6. Post Item → 准备上架

Admin: 完成Post Item（设置售价）

↓

后端: 触发链上更新NFT元数据（价格 + unpause）

↓

JBPM: 等待确认 → 收到信号 → 进入Auction

7. Auction → 拍卖进行

买家: 提交拍卖出价

↓

后端: 记录出价（无链上操作）

↓

JBPM: 买家完成Auction任务 → 进入Auction Review

8. Auction Review → 转移NFT

Admin: 选择中标买家

↓

后端: 触发链上setBuyer + redistribute + transferNFT

↓

JBPM: 等待确认 → 收到信号 → 流程结束

技术架构设计

1. BPMN workflow 修改

需要修改的文件

/Users/fiora/Projects/CC/CollectorCoin/ServerCode/business-central-kjar/src/main/resources/
NewListing.bpmn

修改内容

1. 调整节点顺序: 将Bid Review移到Funding之前
2. 添加等待节点: 在关键区块链操作后添加Intermediate Signal Catch Events

```
<!-- 示例: Value Estimation后的等待节点 -->  
<intermediateCatchEvent id="WaitForNFTMinting" name="Waiting for NFT Minting">  
  <signalEventDefinition signalRef="nftMinted"/>  
</intermediateCatchEvent>
```

需要添加的信号定义

- nftMinted - NFT铸造完成
- investmentConfirmed - 投资确认
- restorationAssigned - 修复分配确认
- restorationCompleted - 修复完成确认
- itemPosted - 物品上架确认
- nftTransferred - NFT转移完成

2. 智能合约层

扩展CCMarketPlace.sol

当前合约只有基础ERC-1155功能，需要添加dNFT管理和事件发出功能。

新增数据结构:

```
enum NFTStatus { CREATED, FUNDING_LIVE, RESTORATION, COMPLETED, AUCTION_LIVE, SOLD }
```

```
struct DNFTMetadata {
    uint256 tokenId;
    address projectAddress;    // 关联CCProject合约
    NFTStatus status;
    uint256 valueEstimation;
    uint256 repairCostEstimation;
    address restorer;
    uint256 auctionPrice;
    bool isPaused;            // 转移控制
    string metadataURI;       // IPFS URI
}
```

```
struct InvestorInfo {
    address[] investors;
    uint256[] amounts;
}
```

核心函数和事件:

```
// 1. 铸造NFT
function mintDNFT(
    address admin,
    address projectAddress,
    uint256 valueEst,
    uint256 repairEst,
    string memory initialMetadataURI
) external onlyOwner returns (uint256) {
    // ... 铸造逻辑 ...
    emit DNFTCreated(newTokenId, projectAddress, admin);
    return newTokenId;
}
```

```
event DNFTCreated(uint256 indexed tokenId, address indexed projectAddress, address admin);
```

// 2. 上架到市场

```
function launchDNFT(uint256 tokenId) external onlyOwner {  
    nftMetadata[tokenId].isLaunched = true;  
    emit DNFTLaunched(tokenId);  
}  
event DNFTLaunched(uint256 indexed tokenId);
```

// 3. 记录投资者

```
function addInvestor(uint256 tokenId, address investor, uint256 amount) external onlyOwner {  
    // ... 添加逻辑 ...  
    emit InvestorAdded(tokenId, investor, amount);  
}  
event InvestorAdded(uint256 indexed tokenId, address investor, uint256 amount);
```

// 4. 设置修复者

```
function updateRestorer(uint256 tokenId, address restorer) external onlyOwner {  
    nftMetadata[tokenId].restorer = restorer;  
    nftMetadata[tokenId].status = NFTStatus.RESTORATION;  
    emit RestorerUpdated(tokenId, restorer);  
}  
event RestorerUpdated(uint256 indexed tokenId, address restorer);
```

// 5. 标记修复完成

```
function markRestorationComplete(uint256 tokenId) external onlyOwner {  
    nftMetadata[tokenId].status = NFTStatus.COMPLETED;  
    emit RestorationCompleted(tokenId);  
}  
event RestorationCompleted(uint256 indexed tokenId);
```

// 6. 更新价格并解除暂停

```
function setItemForSale(uint256 tokenId, uint256 price) external onlyOwner {  
    nftMetadata[tokenId].auctionPrice = price;  
    nftMetadata[tokenId].isPaused = false;  
    emit ItemPosted(tokenId, price);  
}  
event ItemPosted(uint256 indexed tokenId, uint256 price);
```

// 7. 转移NFT给买家

```
function transferToWinner(uint256 tokenId, address winner) external onlyOwner {  
    nftMetadata[tokenId].status = NFTStatus.SOLD;  
    safeTransferFrom(owner(), winner, tokenId, 1, "");  
    emit NFTTransferred(tokenId, winner);  
}  
event NFTTransferred(uint256 indexed tokenId, address winner);
```

// 8. 更新元数据URI

```
function updateMetadataURI(uint256 tokenId, string memory newURI) external onlyOwner {  
    nftMetadata[tokenId].metadataURI = newURI;
```

```
    emit MetadataUpdated(tokenId, newURI);
}
event MetadataUpdated(uint256 indexed tokenId, string metadataURI);
```

转移控制:

使用_beforeTokenTransfer hook检查isPaused标志，防止未授权转移。

3. Java后端层 - 事件监听器

新增核心服务：BlockchainEventListenerService

文件: ServerCode/business-application-service/src/main/java/com/company/services/BlockchainEventListenerService.java

功能: 监听智能合约事件并触发JBPM信号

实现方式: 使用Web3j的Event Filters + Spring @Async

```
@Service
public class BlockchainEventListenerService {

    @Autowired
    private Web3j web3j;

    @Autowired
    private ProcessService processService;

    @Autowired
    private ProjectListingRepository projectListingRepository;

    @Value("${blockchain.contract.marketplace.address}")
    private String marketplaceAddress;

    private static final Logger logger =
        LoggerFactory.getLogger(BlockchainEventListenerService.class);

    /**
     * 启动所有事件监听器
     */
    @PostConstruct
    public void startListeners() {
        listenDNFTCreated();
        listenDNFTLaunched();
        listenInvestorAdded();
        listenRestorerUpdated();
        listenRestorationCompleted();
        listenItemPosted();
        listenNFTTransferred();
    }

    /**
```

```

* 监听DNFTCreated事件
*/
private void listenDNFTCreated() {
    // 事件签名: DNFTCreated(uint256 indexed tokenId, address indexed projectAddress,
address admin)
    Event event = new Event("DNFTCreated",
        Arrays.asList(
            new TypeReference<Uint256>(true) {}, // indexed tokenId
            new TypeReference<Address>(true) {}, // indexed projectAddress
            new TypeReference<Address>() {}      // admin
        )
    );

    EthFilter filter = new EthFilter(
        DefaultBlockParameterName.LATEST,
        DefaultBlockParameterName.LATEST,
        marketplaceAddress
    ).addSingleTopic(EventEncoder.encode(event));

    web3j.ethLogFlowable(filter).subscribe(log -> {
        try {
            // 解析事件参数
            List<String> topics = log.getTopics();
            BigInteger tokenId = new BigInteger(topics.get(1).substring(2), 16);
            String projectAddress = "0x" + topics.get(2).substring(26);

            logger.info("DNFTCreated event received: tokenId={}, projectAddress={}",
                tokenId, projectAddress);

            // 查找对应的ProjectListing
            ProjectListing listing = projectListingRepository
                .findByProjectAddress(projectAddress)
                .orElseThrow(() -> new RuntimeException("Listing not found"));

            // 更新数据库
            listing.setNftTokenId(tokenId.longValue());
            listing.setNftMinted(true);
            projectListingRepository.save(listing);

            // 发送JBPM信号, 推动流程继续
            processService.signalProcessInstance(
                listing.getProcessId(),
                "nftMinted",
                Collections.singletonMap("tokenId", tokenId.longValue())
            );

            logger.info("JBPM signal 'nftMinted' sent for process {}", listing.getProcessId());

        } catch (Exception e) {
            logger.error("Error processing DNFTCreated event", e);
        }
    });
}

```

```

    });
}

/**
 * 监听InvestorAdded事件
 */
private void listenInvestorAdded() {
    // 类似的实现...
    // 当投资者添加成功后，发送"investmentConfirmed"信号
}

/**
 * 监听RestorerUpdated事件
 */
private void listenRestorerUpdated() {
    // 发送"restorationAssigned"信号
}

/**
 * 监听RestorationCompleted事件
 */
private void listenRestorationCompleted() {
    // 发送"restorationCompleted"信号
}

/**
 * 监听ItemPosted事件
 */
private void listenItemPosted() {
    // 发送"itemPosted"信号
}

/**
 * 监听NFTTransferred事件
 */
private void listenNFTTransferred() {
    // 发送"nftTransferred"信号，流程结束
}
}

```

关键技术点:

1. 使用@PostConstruct在应用启动时自动启动监听器
2. 使用web3j.ethLogFlowable()订阅事件流
3. 解析事件日志的topics获取参数
4. 调用processService.signalProcessInstance()发送JBPM信号

4. Java后端层 - 服务修改

新增服务类

NFTMetadataService.java

- generateInitialMetadata() - 生成初始元数据 (Value Estimation时)
- updateWithInvestors() - 添加投资者信息
- updateStatus() - 更新状态和历史记录
- updateAttribute() - 更新特定属性

IPFSService.java

- uploadMetadata() - 上传JSON元数据到IPFS
- fetchMetadata() - 从IPFS获取元数据
- 配置: ipfs.api.url (本地节点或Infura/Pinata API)

元数据JSON结构 (OpenSea标准):

```
{
  "name": "CollectorCoin dNFT #123",
  "description": "1965 Ford Mustang Restoration Project",
  "image": "ipfs://Qm.../image.jpg",
  "external_url": "https://collectorcoin.com/listing/123",
  "attributes": [
    {"trait_type": "VIN", "value": "1FACP..."},
    {"trait_type": "Status", "value": "RESTORATION"},
    {"trait_type": "Value Estimation", "value": "50000", "display_type": "number"},
    {"trait_type": "Investor Count", "value": "8"}
  ],
  "investors": [...],
  "history": [...]
}
```

新增服务: **InvestmentProxyService**

文件: ServerCode/business-application-service/src/main/java/com/company/services/InvestmentProxyService.java

功能: 后端代理执行投资者的链上转账

```
@Service
public class InvestmentProxyService {

    @Autowired
    private BlockchainService blockchainService;

    @Autowired
    private Web3j web3j;

    @Autowired
    private Credentials adminCredentials; // Admin钱包 (gas费支付)
```

```

@Autowired
private ProjectListingRepository projectListingRepository;

/**
 * 代理投资者进行链上投资
 * @param investment 投资信息
 * @param investorAddress 投资者钱包地址
 * @return 交易哈希
 */
@Async
public CompletableFuture<String> proxyInvest(Investment investment, String
investorAddress) {
    try {
        Long listingId = investment.getListing().getId();
        ProjectListing listing = projectListingRepository.findById(listingId)
            .orElseThrow(() -> new RuntimeException("Listing not found"));

        String projectAddress = listing.getProjectAddress();
        BigInteger amount = convertToBigInteger(investment.getAmount());

        // 调用CCProject合约的acceptFunds函数
        // 注意：这里使用Admin钱包代理，需要确保Admin钱包有足够的代币和gas
        Function function = new Function(
            "acceptFunds",
            Arrays.asList(
                new Address(investorAddress), // 投资者地址
                new Uint256(amount)           // 投资金额
            ),
            Collections.emptyList()
        );

        TransactionReceipt receipt = blockchainService.executeTransaction(
            function,
            projectAddress
        );

        String txHash = receipt.getTransactionHash();
        logger.info("Investment proxy transaction sent: {}", txHash);

        // 同时更新NFT的投资者信息
        if(listing.getNftTokenId() != null) {
            blockchainService.addNFTInvestor(
                listing.getNftTokenId(),
                investorAddress,
                amount
            );
        }

        return CompletableFuture.completedFuture(txHash);
    }
}

```

```

        } catch (Exception e) {
            logger.error("Proxy investment failed", e);
            throw new RuntimeException("Proxy investment failed: " + e.getMessage(), e);
        }
    }
}

```

注意事项:

- 使用@Async异步执行，避免阻塞
- Admin钱包需要持有足够的ERC20代币来代理投资
- 需要实现代币授权机制（投资者先授权给后端）
- 或者使用托管账户模式（投资者先充值到平台账户）

扩展现有服务

BlockchainService.java - 新增接口方法:

```

Long mintDNFT(String projectAddress, BigInteger valueEst, BigInteger repairEst, String
metadataURI);
void addNFTInvestor(Long tokenId, String investorAddress, BigInteger amount);
void updateNFTStatus(Long tokenId, int status);
void updateNFTRestorer(Long tokenId, String restorerAddress);
void setNFTAuctionLive(Long tokenId, BigInteger auctionPrice);
void updateNFTMetadataURI(Long tokenId, String newURI);
void transferNFTToWinner(Long tokenId, String winnerAddress);

```

BlockchainServiceImpl.java - 实现方法:

- 使用Web3j的Function和executeTransaction模式
- 从事件日志中提取tokenId（解析DNFTCreated事件）
- 遵循现有的错误处理模式

ProjectListing.java - 新增字段:

```

@Column(name = "nft_token_id")
private Long nftTokenId;

```

```

@Column(name = "nft_metadata_uri")
private String nftMetadataUri;

```

```

@Column(name = "nft_minted")
private Boolean nftMinted = false;

```

```

@Column(name = "nft_launched")
private Boolean nftLaunched = false;

```

```

@Column(name = "nft_transferred")
private Boolean nftTransferred = false;

```

5. 现有服务的异步修改

修改ProjectListingService.java

集成点1: Value Estimation → 异步铸造dNFT

修改: addValueEstimation()方法

从同步:

```
// 旧代码 (同步执行)
blockchainService.saveEstimations(...); // 阻塞等待
listing = projectListingRepository.save(listing);
return listing;
```

改为异步:

```
// 1. 完成JBPM任务 (同步)
Long valueEstimationTaskId = processUtility.getTaskIdByProcessIdAndTitle(...);
processUtility.completeTask(valueEstimationTaskId, "wbadmin", params);

// 2. 异步触发区块链操作 (不等待)
CompletableFuture.runAsync() -> {
    try {
        // 保存估值到CCProject
        blockchainService.saveEstimations(projectAddress, valueEst, repairEst);

        // 生成NFT元数据
        NFTMetadata metadata = nftMetadataService.generateInitialMetadata(listing);
        String metadataURI = ipfsService.uploadMetadata(metadata);

        // 铸造NFT
        blockchainService.mintDNFT(projectAddress, valueEst, repairEst, metadataURI);

        // 上架NFT
        // 注意: tokenId会通过事件监听器获取并保存到数据库
        // 事件触发后会自动发送JBPM信号"nftMinted"

    } catch (Exception e) {
        logger.error("Async NFT minting failed", e);
        // 发送错误通知或回滚机制
    }
};

// 3. JBPM不会立即进入下一任务, 而是等待"nftMinted"信号
// 用户看到的状态: Waiting for NFT Minting...

return listing;
```

集成点2: Funding → 异步代理投资

新建: InvestmentService.createInvestmentWithProxy()方法

```
public Investment createInvestmentWithProxy(Investment investment, Long userId, String investorAddress) {
    // 1. 验证投资金额
    ProjectListing listing = projectListingRepository.findById(investment.getListing().getId())
        .orElseThrow(...);

    if (listing.calcCurrFundingRequiredAmount() < investment.getAmount()) {
        throw new RuntimeException("Investment amount exceeds funding goal");
    }

    // 2. 保存投资记录 (状态: PENDING)
    investment.setStatus(InvestmentStatus.PENDING);
    investment = investmentRepository.save(investment);

    // 3. 异步执行链上投资
    Long investmentId = investment.getId();
    investmentProxyService.proxyInvest(investment, investorAddress)
        .thenAccept(txHash -> {
            logger.info("Investment transaction successful: {}", txHash);
            // 事件监听器会处理后续流程
        })
        .exceptionally(ex -> {
            logger.error("Investment transaction failed", ex);
            // 更新投资状态为FAILED
            Investment failed = investmentRepository.findById(investmentId).get();
            failed.setStatus(InvestmentStatus.FAILED);
            investmentRepository.save(failed);
            return null;
        });

    // 4. 返回 (此时链上交易可能还未确认)
    return investment;
}
```

事件监听器处理:

- InvestorAdded事件触发后, 更新投资状态为CONFIRMED
- 检查是否达到融资目标
- 如果达到, 发送JBPM信号"investmentConfirmed", 推动流程进入Assign Restoration

集成点3: Bid Review → 无NFT操作

修改: BidService.selectBiddingForListing()

```
// Bid Review阶段: 只选择修复者, 不触发区块链操作
public ProjectListing selectBiddingForListing(Long bidId) {
    Bidding bid = biddingRepository.findById(bidId).orElseThrow(...);
```

```

ProjectListing listing = projectListingRepository.findById(bid.getListingId()).orElseThrow(...);

// 设置修复者
listing.setRestorerId(bid.getBidderId());
listing.setBidPrice(bid.getBiddingPrice());

// 完成JBPM任务
processUtility.completeTask(bidReviewTaskId, "wbadmin", params);

// 直接进入Funding任务（无等待）
return listing;
}

```

集成点4: Assign Restoration → 异步分配

修改: ProjectListingService.assignRestoration()

```

// 完成JBPM任务
processUtility.completeTask(assignRestorationTaskId, "wbadmin", params);

// 异步执行链上操作
CompletableFuture.runAsync() -> {
    try {
        // 调用CCProject的assignForRestoration
        blockchainService.assignRestoration(projectAddress);

        // 更新NFT元数据
        NFTMetadata metadata = ipfsService.fetchMetadata(listing.getNftMetadataUri());
        updateAttribute(metadata, "Restorer", restorerAddress);
        String newURI = ipfsService.uploadMetadata(metadata);

        blockchainService.updateRestorer(tokenId, restorerAddress);
        blockchainService.updateMetadataURI(tokenId, newURI);

        // 链上操作完成后，事件监听器会发送"restorationAssigned"信号
    } catch (Exception e) {
        logger.error("Assign restoration failed", e);
    }
});

```

集成点5-8: 其他异步操作（类似模式）

集成点5: Restoration Finished → 异步更新元数据

集成点6: Post Item → 异步设置价格并unpause

集成点7: Auction → 无链上操作（仅记录买家出价）

集成点8: Auction Review → 异步转移NFT和资金再分配

所有这些操作都遵循相同的异步模式：

1. 完成JBPM任务
2. 异步触发区块链操作 (CompletableFuture)
3. JBPM进入等待状态
4. 事件监听器捕获链上事件
5. 发送JBPM信号, 推动流程继续

实施阶段和优先级

⚠️ 架构复杂度提醒

异步事件驱动架构比同步架构复杂3-5倍, 需要更多开发和测试时间。

阶段1: BPMN workflow重构 (1周)

目标: 修改工作流定义, 支持异步信号

任务:

1. ⚠️ 修改BPMN文件: 调整Bid Review和Funding的顺序
2. 添加6个等待节点 (Intermediate Signal Catch Events)
3. 定义6个信号类型
4. 本地JBPM服务器测试信号机制
5. 备份现有BPMN文件

关键文件:

- ServerCode/business-central-kjar/src/main/resources/NewListing.bpmn

验收标准:

- ✅ BPMN在Business Central中无语法错误
- ✅ 可以手动触发信号测试流程流转
- ✅ Bid Review在Funding之前

阶段2: 智能合约开发 (2-3周)

目标: 扩展ERC-1155合约, 添加事件发出

任务:

1. 扩展CCMarketPlace.sol (8个函数 + 8个事件)
2. 编写Solidity单元测试 (Hardhat + Chai)
3. 部署到本地Hardhat网络
4. 验证事件能正确发出
5. Gas费用测试

验收标准:

-  所有函数通过单元测试
-  事件日志可被正确解析
-  转移控制 (isPaused) 正常工作

阶段3: 事件监听器 (2周)

目标: 实现Java端事件监听和JBPM信号触发

任务:

1. 创建BlockchainEventListenerService
2. 实现6个事件监听方法
3. 测试事件捕获和解析
4. 集成JBPM signalProcessInstance
5. 异常处理和重连机制
6. 日志和监控

新建文件:

- BlockchainEventListenerService.java

验收标准:

-  能捕获所有合约事件
-  JBPM信号正确触发
-  监听器在应用重启后自动恢复

阶段4: 后端代理投资 (1-2周)

目标: 实现投资者转账代理服务

任务:

1. 创建InvestmentProxyService
2. 实现代币授权机制
3. 异步执行投资交易
4. 投资状态管理 (PENDING/CONFIRMED/FAILED)
5. 安全审计 (防止重放攻击)

新建文件:

- InvestmentProxyService.java

验收标准:

-  可以代理执行投资交易

-  投资失败时正确回滚
-  Gas费用可承受

阶段5: 服务层异步改造 (2-3周)

目标: 修改现有服务为异步模式

任务:

1. 修改ProjectListingService (6个方法)
2. 修改InvestmentService
3. 修改AuctionService
4. 修改BidService
5. 添加CompletableFuture异步执行
6. 错误处理和超时机制

修改文件:

- ProjectListingService.java
- InvestmentService.java
- AuctionService.java
- BidService.java

验收标准:

-  所有服务方法不阻塞
-  错误情况下有明确的失败处理
-  超时后有重试或告警

阶段6: 元数据和IPFS (1周)

目标: 实现NFT元数据管理

任务:

1. 创建NFTMetadataService
2. 创建IPFSService (本地节点或Pinata)
3. 元数据JSON生成和更新
4. IPFS上传下载
5. 备用方案 (IPFS失败时)

新建文件:

- NFTMetadataService.java
- IPFSService.java
- NFTMetadata.java (DTO)

验收标准:

-  元数据符合OpenSea标准
-  IPFS可正常访问
-  版本历史可追溯

阶段7: 端到端测试 (2-3周)

目标: 完整流程测试

任务:

1. 搭建完整测试环境 (jBPM + Hardhat + IPFS)
2. 测试完整的项目生命周期
3. 压力测试 (并发投资、多项目)
4. 异常场景测试 (网络中断、交易失败)
5. 性能优化
6. 安全审计

验收标准:

-  完整流程无阻塞
-  所有等待节点正确唤醒
-  异常情况有明确提示
-  NFT在OpenSea测试网正确显示

阶段8: 生产部署 (1-2周)

目标: 部署到生产环境

任务:

1. 部署合约到主网或测试网
2. 配置生产环境IPFS
3. 监控和告警设置
4. 文档编写
5. 运维培训

验收标准:

-  生产环境稳定运行
-  完整的监控Dashboard
-  运维文档齐全

预估总时间: 12-18周 (3-4.5个月)

关键文件清单

⚠️ BPMN工作流（修改 - 高风险）

- /Users/fiora/Projects/CC/CollectorCoin/ServerCode/business-central-kjar/src/main/resources/NewListing.bpmn - **重大修改**
 - 调整节点顺序
 - 添加6个等待节点
 - 定义6个信号

智能合约（修改）

- /Users/fiora/Projects/CC/CollectorCoin/ClientCode/tut-react-and-spring-data-rest-master/security/contracts_deployment/contracts/CCMarketPlace.sol - **重大扩展**
 - 8个新函数
 - 8个新事件
 - 转移控制逻辑

Java服务（新建 - 核心）

- ServerCode/business-application-service/src/main/java/com/company/services/BlockchainEventListenerService.java - **最关键**
- ServerCode/business-application-service/src/main/java/com/company/services/InvestmentProxyService.java
- ServerCode/business-application-service/src/main/java/com/company/services/NFTMetadataService.java
- ServerCode/business-application-service/src/main/java/com/company/services/IPFSService.java
- ServerCode/business-application-service/src/main/java/com/company/dto/NFTMetadata.java
- ServerCode/business-application-service/src/main/java/com/company/dto/Attribute.java

Java服务（修改 - 异步改造）

- /Users/fiora/Projects/CC/CollectorCoin/ServerCode/business-application-service/src/main/java/com/company/services/BlockchainService.java - 新增NFT方法接口
- /Users/fiora/Projects/CC/CollectorCoin/ServerCode/business-application-service/src/main/java/com/company/services/BlockchainServiceImpl.java - 实现NFT方法
- /Users/fiora/Projects/CC/CollectorCoin/ServerCode/business-application-service/src/main/java/com/company/services/ProjectListingService.java - **异步改造**
- /Users/fiora/Projects/CC/CollectorCoin/ServerCode/business-application-service/src/main/java/com/company/services/InvestmentService.java - **异步改造**

- /Users/fiora/Projects/CC/CollectorCoin/ServerCode/business-application-service/src/main/java/com/company/services/AuctionService.java - 异步改造
- /Users/fiora/Projects/CC/CollectorCoin/ServerCode/business-application-service/src/main/java/com/company/services/BidService.java - 修改顺序

数据模型（修改）

- /Users/fiora/Projects/CC/CollectorCoin/ServerCode/business-application-model/src/main/java/com/company/models/ProjectListInfo.java - 新增NFT字段
- ServerCode/business-application-model/src/main/java/com/company/models/Investment.java - 新增状态字段

配置文件（修改）

- ServerCode/business-application-service/src/main/resources/application.properties

技术考虑

元数据更新策略

IPFS版本链 + 链上指针:

- 每次更新生成新的IPFS哈希
- 链上只存储最新URI（节省Gas）
- 所有历史版本仍可访问（完整追溯）

错误处理策略

分层错误处理:

操作	失败影响	处理策略
jBPM任务	关键	抛出异常，回滚
CCProject链上操作	关键	抛出异常，阻塞流程
NFT铸造	中等	记录错误，允许继续
元数据更新	轻微	记录警告，不阻塞

Gas费优化

1. 批量操作: 每5笔投资更新一次链上数据
2. 延迟更新: 非关键元数据变更可延迟到下一个主要阶段
3. 链下为主: 详细信息存IPFS，链上只存关键状态

数据一致性

- CCProject合约：资金流和投资逻辑
- CCMarketPlace合约：NFT和元数据
- ProjectListing数据库：真实来源，定期验证同步

配置示例

application.properties

```
# NFT Marketplace Contract
blockchain.contract.marketplace.address=0x...

# IPFS Configuration
ipfs.api.url=http://localhost:5001/api/v0
ipfs.gateway.url=https://ipfs.io/ipfs/
ipfs.enabled=true

# NFT Metadata
nft.metadata.base.image.url=ipfs://Qm.../default.jpg
nft.metadata.external.base.url=https://collectorcoin.com/listing/
```

风险和缓解

风险	影响	缓解措施
----- ----- -----		
BPMN修改破坏现有流程	⚠️ 极高	1. 完整备份现有BPMN2. 在测试环境充分验证3. 保留回滚方案
事件监听器失败	极高	1. 实现心跳检测2. 自动重连机制3. 告警系统4. 手动恢复工具
异步状态不一致	极高	1. 实现状态机管理2. 超时检测3. 手动干预接口
JBPM信号未触发	极高	1. 记录所有信号发送2. 定期检查挂起的流程3. 手动触发工具
投资代理安全风险	极高	1. 严格的授权验证2. 金额上限控制3. 多签钱包4. 安全审计
IPFS节点不可用	高	使用Pinata/Infura备用服务
Gas费过高	中	批量操作，L2方案
并发投资冲突	中	乐观锁，事务隔离

成功标准

功能性标准

- ✅ 所有8个集成点异步执行成功
- ✅ JBPM工作流在异步模式下正确流转
- ✅ 所有等待节点能被正确唤醒
- ✅ 事件监听器稳定运行，无遗漏
- ✅ NFT在OpenSea等市场正确显示

6.  投资代理安全且可靠
7.  拍卖结束后NFT正确转移给买家

非功能性标准

8.  事件监听延迟 < 30秒
9.  异常情况下有明确的告警
10.  系统可自动从故障中恢复
11.  Gas费用在预算范围内
12.  完整的日志和监控Dashboard
13.  测试覆盖率 > 70%

关键决策点

在开始实施前，请再次确认：

1. 是否接受工作流顺序变更？（Bid Review移到Funding之前）
 - 这需要修改BPMN文件
 - 可能影响现有用户体验
2. 是否接受后端代理投资？
 - 需要托管用户资金或实现授权机制
 - 有一定的安全风险
3. 是否接受3-4.5个月的开发时间？
 - 异步架构比同步复杂很多
 - 需要充分的测试时间
4. 团队是否有区块链事件监听经验？
 - 这是整个系统的关键技术点
 - 建议先做技术验证（POC）

下一步行动

建议：先做技术验证（POC）

在全面实施前，建议用1-2周时间做技术验证：

1. 验证1: JBPM信号机制
 - 修改一个简单的BPMN流程
 - 添加一个等待节点
 - 手动发送信号测试
2. 验证2: 事件监听器
 - 部署一个简单的测试合约

- 实现一个事件监听器
- 验证能稳定捕获事件

3. **验证3:** 异步执行

- 实现一个简单的异步区块链调用
- 验证CompletableFuture工作正常

如果POC成功，再按照阶段1开始全面实施。